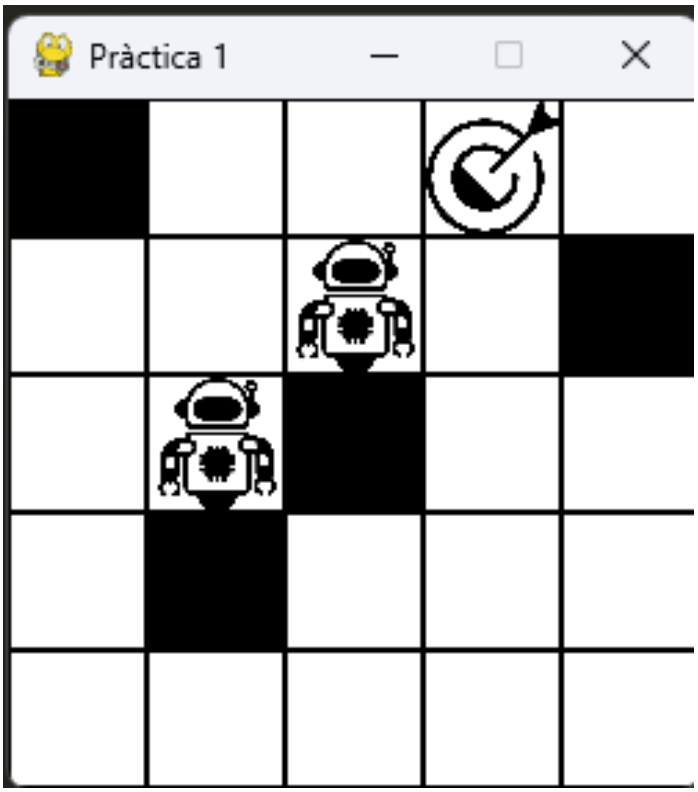
PRÀCTICA 1 - IA



Autors: Joan Gelabert Oliver i Xavier Estades Novo

Aquesta pràctica ha estat feta amb ajuda de ChatGPT 4 i DeepSeek en la creació de codi i en la revisió gramatical

Algoritme BFS **3**
Algoritme A* **3**
Algoritme MiniMax amb poda alpha-beta **5**
1. Classe Agent Minimax **5**
Objectiu general 5
Components clau 5
2. Classe Estat **7**
Objectiu general 7
Components clau 7
Observacions i raonaments generals 8

Algoritme BFS

Aquest agent implementa una cerca en amplada (BFS) per trobar el camí més curt entre la seva posició i el destí dins un laberint. A cada iteració de la funció **actua()**, l'agent comprova si ja té un camí previ (self.camí); si no, executa el BFS per generar-lo.

L'algoritme utilitza una cua FIFO (deque) que emmagatzema tuples amb la posició actual i el camí recorregut fins a ella. També fa ús d'un conjunt de visitats (set) per evitar repetir posicions i, així, reduir exploracions innecessàries. A cada pas, s'exploren totes les possibles direccions definides a **joc.Laberint.MOVS**, comprovant que les noves coordenades siguin dins dels límits i no coincideixin amb parets. Quan s'arriba al destí, el camí trobat s'assigna a self.camí i, a les següents iteracions, l'agent simplement segueix aquestes instruccions pas a pas mitjançant "MOURE", self.camí.pop(0).

Complexitat temporal:
L'algoritme visita totes les caselles com a màxim una vegada i cada moviment genera fins a quatre successors (N, O, S, E). Per tant, la complexitat és O(V + E), que s'aproxima a O(V), on V és el nombre total de caselles del laberint (files × columnes).

En conclusió, l'agent és complet i òptim, ja que sempre troba el camí més curt si existeix. No obstant això, pot arribar a ser costós en temps i memòria en laberints grans, ja que BFS explora molts estats abans d'arribar al destí.

Algoritme A*

Aquest agent implementa una versió de la cerca informada A* per trobar el camí més eficient cap al destí dins un laberint. L'objectiu és millorar la cerca exhaustiva del BFS afegint una heurística que prioritza els estats més prometedors, reduint el nombre de nodes explorats.

L'agent fa servir dues estructures de dades principals:

- g: el cost real des de l'origen fins a cada node.
- f: el cost total estimat, definit com f(n) = g(n) + h(n), on h(n) és la heurística. Aquests valors s'emmagatzemen dins un defaultdict amb valor inicial infinit per simplificar la gestió dels costos.

La heurística utilitzada és la distància Manhattan, definida com:

$$h(n) = |x_1 - x_2| + |y_1 - y_2|,$$

on x_1 i y_1 són les coordenades de la posició inicial de l'agent i x_2 i y_2 són les coordenades de la posició de la meta.

Aquesta mesura és adequada per entorns de graella on no hi ha moviments diagonals (N, O, S, E) . És admissible i consistent, garantint així que A* trobi el camí òptim.
La funció genera_successors() construeix els possibles moviments a partir d'una posició. Genera tres tipus d'accions:

1. BOTAR (salt de dues caselles) amb cost 2.
2. MOURE (una casella adjacent) amb cost 1.
3. POSAR PARET amb cost 3.

Aquesta jerarquia (BOTAR > MOURE > POSAR PARET) estableix una preferència d'accions dins la cerca on el sistema tendeix a prioritzar salts més ràpids i útils abans que moviments normals, i només opta per col·locar parets si no hi ha cap alternativa millor. Aquesta diferència de costos afecta directament el càlcul del valor f(n) i fa que l'algoritme esculli camins amb accions més barates.
Durant l'execució, es fa servir una cua de prioritat (PriorityQueue) per seleccionar el node amb menor f(n) a cada pas. Si s'arriba al destí, el camí es reconstrueix retrocedint des del node final fins a l'inicial mitjançant el diccionari pare.

Complexitat temporal:
En el pitjor cas, és O(E) (tots els nodes i arestes explorats), però en la pràctica és molt més eficient que BFS, ja que l'heurística guia la cerca cap al destí i evita expansions innecessàries.

Detalls clau del codi:

- defaultdict(lambda: math.inf) permet inicialitzar automàticament els costos a infinit.
- L'ús de PriorityQueue és correcte per ordenar els estats segons el valor f(n).
- El conjunt visitats evita processar el mateix node més d'una vegada.
- La reconstrucció del camí es fa invertint l'ordre d'accions (reverse()), per recuperar la seqüència correcta.

En conclusió, l'agent A* és òptim i complet si l'heurística és admissible i consistent. La distància Manhattan és una elecció encertada per aquest tipus de laberint. L'algoritme és més ràpid que BFS i redueix considerablement la memòria utilitzada, ja que centra la cerca en la direcció del destí.

Algoritme MiniMax amb poda alpha-beta

1. Classe Agent Minimax

Objectiu general

Aquesta classe defineix un agent capaç de prendre decisions en un entorn de laberint amb diversos agents i un destí fix. L'agent utilitza una estratègia de cerca basada en Minimax amb poda alfa-beta per determinar l'acció òptima a realitzar. Aquesta estratègia permet a l'agent:

- Anticipar els moviments dels altres agents.
- Prioritzar moviments que l'acosten al destí.
- Evitar estats desfavorables i bloquejos.

Components clau

1. **Constants d'avaluació**
   L'agent utilitza valors heurístics per assignar importància a situacions concretes:
   - Una recompensa elevada si arriba al destí.
   - Una penalització si un altre agent arriba primer.
   - Un petit avantatge si és el torn de l'agent.
   - Una penalització basada en la distància al destí per prioritzar moviments eficients.
2. **Inicialització**
   L'agent s'inicialitza amb informació sobre el seu nom, les dimensions del taulell i la profunditat màxima de cerca. També manté un mecanisme intern de memòria per emmagatzemar resultats previs de l'avaluació d'estats, amb l'objectiu d'optimitzar l'execució del Minimax.

3. **Mètode principal de decisió**
   L'agent avalua l'estat actual i determina:

    - Si ja ha arribat al destí.

    - Si un altre agent ha arribat al destí.

    - Si és el torn de l'agent i ha de maximitzar la seva puntuació.
      Posteriorment, utilitza Minimax amb poda alfa-beta per seleccionar la millor acció disponible segons l'estat i les possibles respostes dels altres agents.

4. **Construcció de l'estat intern**
   La percepció de l'entorn es transforma en un objecte d'estat complet que representa:

    - Posicions de tots els agents.

    - Parets i obstacles existents.

    - Posició del destí.

    - Torn actual i dimensions del taulell.
      Aquesta representació uniforme permet que l'agent pugui operar de manera recursiva sense dependre del sistema extern.

5. **Estratègia Minimax amb poda alfa-beta**

    - Avalua de manera recursiva els estats successors possibles.

    - Decideix entre maximitzar o minimitzar segons si és el torn de l'agent o d'un contrari.

    - Evita recalcular estats ja explorats gràcies a la memòria interna.

    - Utilitza la poda alfa-beta per reduir significativament la quantitat d'estats que s'han de valorar, millorant l'eficiència.

6. **Funció d'avaluació heurística**

    - Assigna un valor a cada estat segons:

        - Si l'agent ha guanyat o perdut.

        - La distància a la meta.

        - L'avantatge temporal de ser l'agent que té el torn.

- Permet al Minimax prioritzar estats més favorables sense necessitat de simular tot el joc fins al final.

---

2. Classe Estat

Objectiu general

Representa la configuració d'un moment concret del joc, incloent posicions dels agents, parets, destí i torn actual. Aquesta representació és essencial per generar els possibles moviments de manera coherent i permetre l'operació del Minimax.

Components clau

1. **Validació i accessibilitat**
    - Cada posició es comprova per assegurar que està dins del taulell.
    - També es verifica que no hi hagi obstacles ni altres agents en la posició objectiu.
    - Aquesta comprovació garanteix que els moviments generats siguin sempre vàlids.
2. **Generació d'estats successors**
   L'estat pot produir tots els possibles successors per un agent en torn. Les accions contemplades inclouen:
    - Moure's a una casella adjacent.
    - Saltar dues caselles en línia recta.
    - Col·locar una paret a una casella adjacent.
      Aquesta generació és la base de l'arbre de decisions que utilitza Minimax.
3. **Gestió de torns**
   Permet determinar quin agent actuarà després del torn actual, assegurant un ordre correcte en la simulació del joc.
4. **Determinació d'estats terminals**
    - Un estat és terminal si algún agent ha arribat al destí.

    - També és terminal si no existeixen moviments possibles per l'agent en torn. Aquesta informació és crucial per Minimax, ja que els estats terminals tenen un valor heurístic definit i interrompen la recursió.

5. **Heurística simple**
   Calcula la distància Manhattan de l'agent al destí com a mesura de proximitat i eficiència en el moviment. Aquesta heurística és útil com a guia per a la recerca i per optimitzar l'estratègia.

Observacions i raonaments generals

- L'arquitectura separa clarament la **lògica de decisió** (Agent Minimax Viatger) de la **representació de l'entorn** (Estat). Aquesta separació facilita la reutilització i el testatge.